

WEB ARCHITECTURE

Frontend, Backend, Databases & Beyond

A practical map for HTML / CSS / JS learners — plus an intro to Flask, Python, and Git & GitHub

Frontend

Backend

Database

Flask + Python

Git & GitHub

What We'll Cover

1

The Big Picture

How the web actually works, end to end

3

Backend

Server logic + frameworks across languages

5

The Extra Pieces

APIs, servers, hosting, DNS, CDNs

7

Flask + Python

A gentle first backend framework

2

Frontend

What it is + the major frameworks to know

4

Database

SQL vs NoSQL, and where data lives

6

Your Roadmap

From HTML/CSS/JS to full-stack, step by step

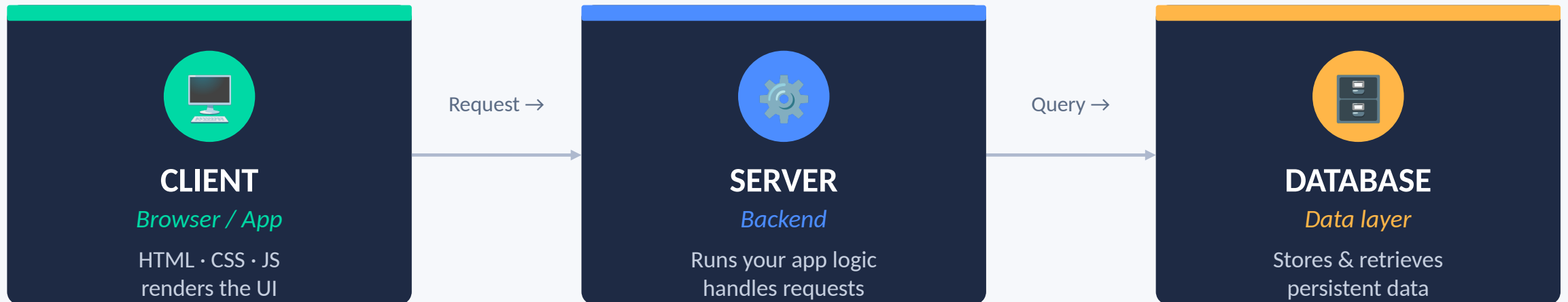
8

Git & GitHub

Version control basics every dev needs

Every Website Is a Conversation

A browser (client) asks a server for something. The server figures out the answer — sometimes by asking a database — and sends it back.



Response ← flows back the same path: Database → Server → Client

Frontend: Everything the User Sees & Touches



HTML

Structure — the skeleton of every page (headings, text, images, forms)



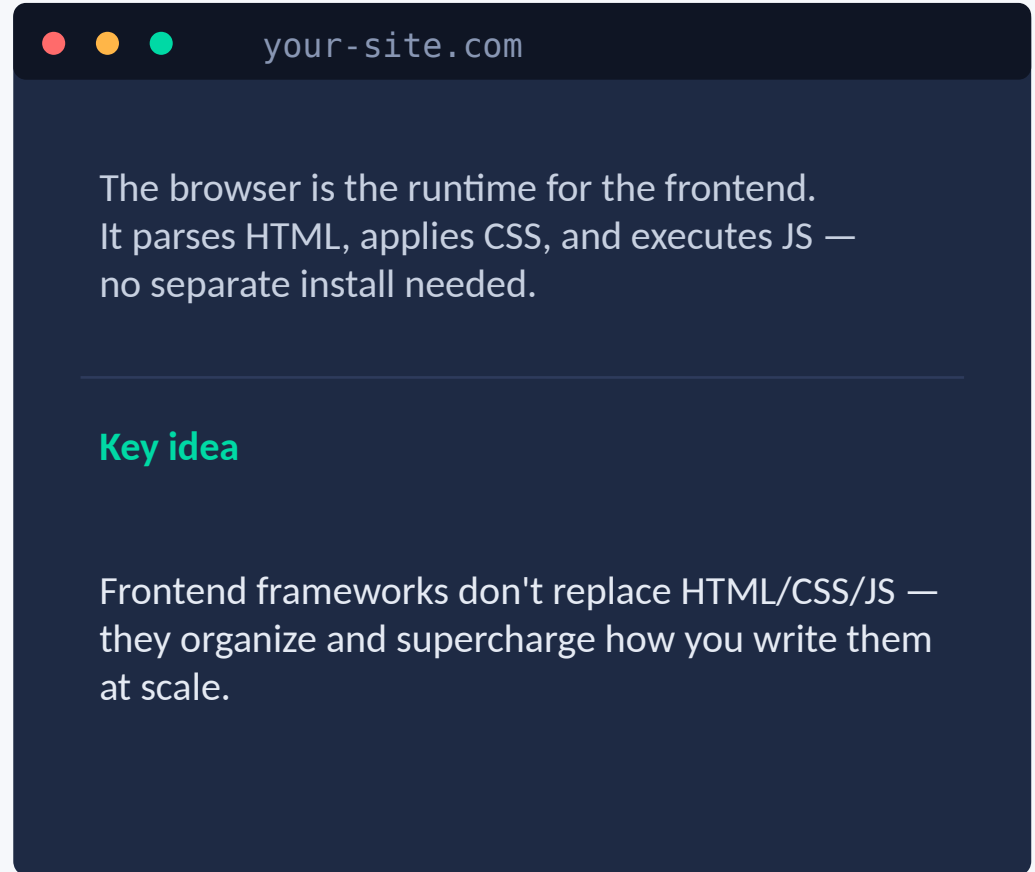
CSS

Style — layout, color, spacing, responsiveness, animation



JavaScript

Behavior — interactivity, DOM updates, talking to the backend



Frontend Frameworks & Libraries

Tools that help you build UIs faster and manage complex state

React

LIBRARY

Component-based UI, huge ecosystem, backed by Meta

Vue

FRAMEWORK

Gentle learning curve, great docs, flexible adoption

Angular

FRAMEWORK

Full-featured, opinionated, popular in enterprises

Svelte

COMPILER

Compiles away at build time — less runtime code

Next.js

META-FRAMEWORK

React + routing, SSR, and backend in one

Tailwind CSS

CSS TOOLKIT

Utility-first styling instead of hand-written CSS

Backend: The Engine Room

Code that runs on a server, invisible to the user, doing the real work behind every click.

1

Business logic

Rules that decide what the app should do (e.g. "can this user checkout?")

2

Authentication

Verifying who a user is, and what they're allowed to access

3

APIs

Endpoints the frontend calls to fetch or send data

4

Talking to the database

Reading, writing, and updating stored data safely



```
GET /api/products
```

```
→ check auth token
```

```
→ query database
```

```
→ format JSON response
```

```
200 OK { products: [...] }
```

This whole exchange happens in milliseconds — the user just sees the page update.

Backend Frameworks, by Language

Same job — handle requests, run logic, talk to the database — different languages

Python	Flask, Django, FastAPI
JavaScript / Node.js	Express, NestJS
Ruby	Ruby on Rails
Java / Kotlin	Spring Boot
PHP	Laravel
Go	Gin, Fiber

We'll zoom into Flask (Python) shortly — a great first backend framework.

Database: Where Data Lives

Without a database, every user, post, and order would vanish the moment the server restarts.

1

Store

Save data permanently — users, orders, messages, files

2

Retrieve

Fetch exactly the data needed, fast, even at huge scale

3

Update & Delete

Keep data accurate as things change over time

4

Protect

Control who can read or write which data

users

id	name	email
1	Asha K.	asha@ex.com
2	Ravi P.	ravi@ex.com
3	Meera S.	meera@ex.com

This is what a relational (SQL) table looks like — rows and columns, just like a spreadsheet.

Types of Databases: SQL vs NoSQL

SQL (Relational)

Structured, tables with fixed columns

- Data organized into rows & tables
- Great for related, structured data
- Uses SQL query language
- Strong consistency guarantees

EXAMPLES

PostgreSQL · MySQL · SQLite · MS SQL Server

NoSQL (Non-relational)

Flexible, document/key-based structure

- Data as documents, key-value, or graphs
- Great for flexible or fast-changing data
- Scales horizontally with ease
- Schema can evolve without migrations

EXAMPLES

MongoDB · Redis · Firebase · DynamoDB

The Other Pieces of the Puzzle

A site is more than frontend + backend + database — these pieces make it reachable and fast.

A API

The contract frontend & backend use to exchange data (often REST or GraphQL)

W Web Server

Software (Nginx, Apache) that receives requests and routes them

H Hosting / Cloud

Where your app actually runs — AWS, Vercel, Render, Heroku

D DNS

Translates a domain name (example.com) into a server's IP address

C CDN

Caches static files closer to users worldwide for speed

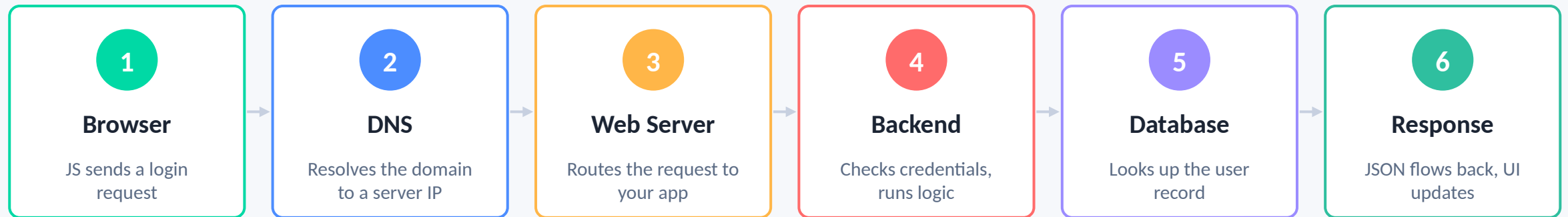
A Authentication

Confirms who a user is before granting access

PUTTING IT TOGETHER

How a Single Click Travels

Example: you click "Sign In" on a website



All of this typically happens in well under a second.

From HTML/CSS/JS to Full-Stack

A realistic, step-by-step order — no need to learn everything at once

1

Solidify JS fundamentals

DOM, fetch/APIs, async/await, ES6+ — before any framework

2

Learn one frontend framework

React is the most in-demand; Vue is the gentlest start

3

Learn backend basics

HTTP, routes, request/response — then pick Flask or Express

4

Learn database basics

SQL fundamentals first, then explore a NoSQL option

5

Learn Git & GitHub

Version control from day one — not an afterthought

6

Build & deploy real projects

Ship small full-stack apps; deploy on Render/Vercel/Heroku

Don't skip the fundamentals

Frameworks change every few years. Strong HTML, CSS, JS, and problem-solving skills outlast every trend and make every framework easier to pick up.

One framework at a time

Pick one frontend and one backend framework to go deep on before comparing alternatives. Breadth comes later.



NEXT UP

Intro to Flask & Python

A beginner-friendly first step into backend development

What Is Flask?

Flask is a lightweight ("micro") web framework for Python — it gives you just enough structure to build a backend, without forcing a rigid setup.



Minimal & unopinionated

You choose your own tools for database, auth, etc.



Easy to learn

A working app in ~10 lines of Python



Great for APIs & small-medium apps

Perfect for learning and prototyping



Backed by a huge Python ecosystem

Pairs with any library you already know



app.py

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run(debug=True)
```

That's a complete, running web server.

Flask: Core Concepts to Learn First

Routes

```
@app.route('/path')
```

Maps a URL to a Python function

Templates (Jinja2)

```
render_template('page.html')
```

Injects Python data into HTML files

Request handling

```
request.form / request.args
```

Reads data sent from forms or the URL

Static files

```
/static/style.css
```

Serves CSS, JS, and images

Flask extensions

```
Flask-SQLAlchemy, Flask-Login
```

Add a database, auth, and more as needed

Why Python here?

```
Readable syntax
```

Same language for scripting, data & backend

Git & GitHub: Save Points for Your Code

Git

A version control tool that runs on your computer

- Tracks every change to your files over time
- Lets you undo mistakes and compare versions
- Enables branches — safe spaces to experiment
- Works completely offline

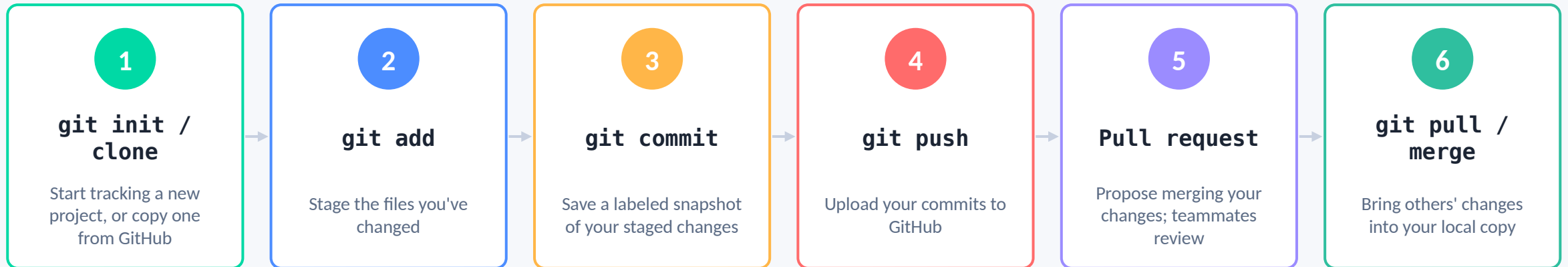
GitHub

A cloud platform for hosting Git repositories

- Backs up your code off your machine
- Lets teams collaborate on the same project
- Pull requests enable code review before merging
- Doubles as a portfolio recruiters actually check

The Everyday Git Workflow

The handful of commands that cover 90% of daily use



Tip: commit small, commit often — write messages that explain "why", not just "what".

Key Takeaways

- 1 Frontend, backend, and database are three distinct jobs
- 2 Frameworks organize HTML/CSS/JS & backend languages — they don't replace fundamentals
- 3 APIs, hosting, DNS, and CDNs are what make a site reachable and fast
- 4 Flask is a friendly, minimal first backend framework in Python
- 5 Git & GitHub aren't optional extras — they're core developer habits from day one

ACTION ITEMS

Your To-Do List After Today



Install Python, VS Code & Git

Set up your local dev environment before anything else



Follow the official Flask Quickstart

flask.palletsprojects.com — build a tiny app end-to-end



Pick ONE frontend framework to try

React or Vue — build one small project with it



Deploy one project publicly

Render, Vercel, or PythonAnywhere — see it live on the internet



Build 3 static pages with HTML/CSS/JS

Cement fundamentals before touching a framework



Create a GitHub account & first repo

Push your static pages as commit #1



Learn basic SQL

SELECT, INSERT, UPDATE, JOIN — enough to query a real database



Read "How the Web Works" style guides

MDN Web Docs is the best free reference, bookmark it